```
template <typename HashedObj, typename Object>
class Pair
{
    HashedObj key;
    Object    def;
    // Appropriate Constructors, etc.
};

template <typename HashedObj, typename Object>
class Dictionary
{
  public:
    Dictionary( );

    void insert( const HashedObj & key, const Object & definition );
    const Object & lookup( const HashedObj & key ) const;
    bool isEmpty( ) const;
    void makeEmpty( );

  private:
    HashTable<Pair<HashedObj,Object>> items;
};
```

Figure 5.56 Dictionary skeleton for Exercise 5.20

5.24 Implement a hopscotch hash table and compare its performance with linear probing, separate chaining, and cuckoo hashing.

5.25 Implement the classic cuckoo hash table in which two separate tables are maintained. The simplest way to do this is to use a single array and modify the hash function to access either the top half or the bottom half.

5.26 Extend the classic cuckoo hash table to use d hash functions.

5.27 Show the result of inserting the keys 10111101, 00000010, 10011011, 10111110, 01111111, 01010001, 10010110, 00001011, 11001111, 10011110, 11011011, 00101011, 01100001, 11110000, 01101111 into an initially empty extendible hashing data structure with $M = 4$.

5.28 Write a program to implement extendible hashing. If the table is small enough to fit in main memory, how does its performance compare with separate chaining and open addressing hashing?

References

Despite the apparent simplicity of hashing, much of the analysis is quite difficult, and there are still many unresolved questions. There are also many interesting theoretical issues.

Hashing dates to at least 1953, when H. P. Luhn wrote an internal IBM memorandum that used separate chaining hashing. Early papers on hashing are [11] and [32]. A wealth of information on the subject, including an analysis of hashing with linear probing under the assumption of totally random and independent hashing, can be found in [25]. More recent results have shown that linear probing requires only 5-independent hash functions [31]. An excellent survey on early classic hash tables methods is [28]; [29] contains suggestions, and pitfalls, for choosing hash functions. Precise analytic and simulation results for separate chaining, linear probing, quadratic probing, and double hashing can be found in [19]. However, due to changes (improvements) in computer architecture and compilers, simulation results tend to quickly become dated.

An analysis of double hashing can be found in [20] and [27]. Yet another collision resolution scheme is coalesced hashing, described in [33]. Yao [37] has shown the uniform hashing, in which no clustering exists, is optimal with respect to cost of a successful search, assuming that items cannot move once placed.

Universal hash functions were first described in [5] and [35]; the latter paper introduces the "Carter-Wegman trick" of using Mersenne prime numbers to avoid expensive mod operations. Perfect hashing is described in [16], and a dynamic version of perfect hashing was described in [8]. [12] is a survey of some classic dynamic hashing schemes.

The $\Theta(\log N/\log\log N)$ bound on the length of the longest list in separate chaining was shown (in precise form) in [18]. The "power of two choices," showing that when the shorter of two randomly selected lists is chosen, then the bound on the length of the longest list is lowered to only $\Theta(\log\log N)$, was first described in [2]. An early example of the power of two choices is [4]. The classic work on cuckoo hashing is [30]; since the initial paper, a host of new results have appeared that analyze the amount of independence needed in the hash functions and describe alternative implementations [7], [34], [15], [10], [23], [24], [1], [6], [9] and [17]. Hopscotch hashing appeared in [21].

Extendible hashing appears in [13], with analysis in [14] and [36].

Exercise 5.15 (a–d) is from [22]. Part (e) is from [26], and part (f) is from [3]. The FNV-1a hash function described in Exercise 5.9 is due to Fowler, Noll, and Vo.

1. Y. Arbitman, M. Naor, and G. Segev, "De-Amortized Cuckoo Hashing: Provable Worst-Case Performance and Experimental Results," *Proceedings of the 36th International Colloquium on Automata, Languages and Programming* (2009), 107–118.
2. Y. Azar, A. Broder, A. Karlin, and E. Upfal, "Balanced Allocations," *SIAM Journal of Computing*, 29 (1999), 180–200.
3. R. S. Boyer and J. S. Moore, "A Fast String Searching Algorithm," *Communications of the ACM,* 20 (1977), 762–772.
4. A. Broder and M. Mitzenmacher, "Using Multiple Hash Functions to Improve IP Lookups," *Proceedings of the Twentieth IEEE INFOCOM* (2001), 1454–1463.
5. J. L. Carter and M. N. Wegman, "Universal Classes of Hash Functions," *Journal of Computer and System Sciences,* 18 (1979), 143–154.
6. J. Cohen and D. Kane, "Bounds on the Independence Required for Cuckoo Hashing," preprint.
7. L. Devroye and P. Morin, "Cuckoo Hashing: Further Analysis," *Information Processing Letters*, 86 (2003), 215–219.

8. M. Dietzfelbinger, A. R. Karlin, K. Melhorn, F. Meyer auf der Heide, H. Rohnert, and R. E. Tarjan, "Dynamic Perfect Hashing: Upper and Lower Bounds," *SIAM Journal on Computing,* 23 (1994), 738–761.
9. M. Dietzfelbinger and U. Schellbach, "On Risks of Using Cuckoo Hashing with Simple Universal Hash Classes," *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms* (2009), 795–804.
10. M. Dietzfelbinger and C. Weidling, "Balanced Allocation and Dictionaries with Tightly Packed Constant Size Bins," *Theoretical Computer Science*, 380 (2007), 47–68.
11. I. Dumey, "Indexing for Rapid Random-Access Memory," *Computers and Automation*, 5 (1956), 6–9.
12. R. J. Enbody and H. C. Du, "Dynamic Hashing Schemes," *Computing Surveys,* 20 (1988), 85–113.
13. R. Fagin, J. Nievergelt, N. Pippenger, and H. R. Strong, "Extendible Hashing—A Fast Access Method for Dynamic Files," *ACM Transactions on Database Systems,* 4 (1979), 315–344.
14. P. Flajolet, "On the Performance Evaluation of Extendible Hashing and Trie Searching," *Acta Informatica,* 20 (1983), 345–369.
15. D. Fotakis, R. Pagh, P. Sanders, and P. Spirakis, "Space Efficient Hash Tables with Worst Case Constant Access Time," *Theory of Computing Systems*, 38 (2005), 229–248.
16. M. L. Fredman, J. Komlos, and E. Szemeredi, "Storing a Sparse Table with $O(1)$ Worst Case Access Time," *Journal of the ACM,* 31 (1984), 538–544.
17. A. Frieze, P. Melsted, and M. Mitzenmacher, "An Analysis of Random-Walk Cuckoo Hashing," *Proceedings of the Twelfth International Workshop on Approximation Algorithms in Combinatorial Optimization (APPROX)* (2009), 350–364.
18. G. Gonnet, "Expected Length of the Longest Probe Sequence in Hash Code Searching," *Journal of the Association for Computing Machinery*, 28 (1981), 289–304.
19. G. H. Gonnet and R. Baeza-Yates, *Handbook of Algorithms and Data Structures,* 2d ed., Addison-Wesley, Reading, Mass., 1991.
20. L. J. Guibas and E. Szemeredi, "The Analysis of Double Hashing," *Journal of Computer and System Sciences,* 16 (1978), 226–274.
21. M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch Hashing," *Proceedings of the Twenty-Second International Symposium on Distributed Computing* (2008), 350–364.
22. R. M. Karp and M. O. Rabin, "Efficient Randomized Pattern-Matching Algorithms," *Aiken Computer Laboratory Report TR-31-81,* Harvard University, Cambridge, Mass., 1981.
23. A. Kirsch and M. Mitzenmacher, "The Power of One Move: Hashing Schemes for Hardware," *Proceedings of the 27th IEEE International Conference on Computer Communications (INFOCOM)* (2008), 106–110.
24. A. Kirsch, M. Mitzenmacher, and U. Wieder, "More Robust Hashing: Cuckoo Hashing with a Stash," *Proceedings of the Sixteenth Annual European Symposium on Algorithms* (2008), 611–622.
25. D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching,* 2d ed., Addison-Wesley, Reading, Mass., 1998.
26. D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast Pattern Matching in Strings," *SIAM Journal on Computing,* 6 (1977), 323–350.
27. G. Lueker and M. Molodowitch, "More Analysis of Double Hashing," *Proceedings of the Twentieth ACM Symposium on Theory of Computing* (1988), 354–359.
28. W. D. Maurer and T. G. Lewis, "Hash Table Methods," *Computing Surveys,* 7 (1975), 5–20.

29. B. J. McKenzie, R. Harries, and T. Bell, "Selecting a Hashing Algorithm," *Software—Practice and Experience,* 20 (1990), 209–224.
30. R. Pagh and F. F. Rodler, "Cuckoo Hashing," *Journal of Algorithms*, 51 (2004), 122–144.
31. M. Pătraşcu and M. Thorup, "On the *k*-Independence Required by Linear Probing and Minwise Independence," *Proceedings of the 37th International Colloquium on Automata, Languages, and Programming* (2010), 715–726.
32. W. W. Peterson, "Addressing for Random Access Storage," *IBM Journal of Research and Development,* 1 (1957), 130–146.
33. J. S. Vitter, "Implementations for Coalesced Hashing," *Communications of the ACM,* 25 (1982), 911–926.
34. B. Vöcking, "How Asymmetry Helps Load Balancing," *Journal of the ACM*, 50 (2003), 568–589.
35. M. N. Wegman and J. Carter, "New Hash Functions and Their Use in Authentication and Set Equality," *Journal of Computer and System Sciences*, 22 (1981), 265–279.
36. A. C. Yao, "A Note on the Analysis of Extendible Hashing," *Information Processing Letters,* 11 (1980), 84–86.
37. A. C. Yao, "Uniform Hashing Is Optimal," *Journal of the ACM,* 32 (1985), 687–693.

CHAPTER 6

Priority Queues (Heaps)

Although jobs sent to a printer are generally placed on a queue, this might not always be the best thing to do. For instance, one job might be particularly important, so it might be desirable to allow that job to be run as soon as the printer is available. Conversely, if, when the printer becomes available, there are several 1-page jobs and one 100-page job, it might be reasonable to make the long job go last, even if it is not the last job submitted. (Unfortunately, most systems do not do this, which can be particularly annoying at times.)

Similarly, in a multiuser environment, the operating system scheduler must decide which of several processes to run. Generally, a process is allowed to run only for a fixed period of time. One algorithm uses a queue. Jobs are initially placed at the end of the queue. The scheduler will repeatedly take the first job on the queue, run it until either it finishes or its time limit is up, and place it at the end of the queue if it does not finish. This strategy is generally not appropriate, because very short jobs will seem to take a long time because of the wait involved to run. Generally, it is important that short jobs finish as fast as possible, so these jobs should have precedence over jobs that have already been running. Furthermore, some jobs that are not short are still very important and should also have precedence.

This particular application seems to require a special kind of queue, known as a **priority queue.** In this chapter, we will discuss . . .

- Efficient implementation of the priority queue ADT.
- Uses of priority queues.
- Advanced implementations of priority queues.

The data structures we will see are among the most elegant in computer science.

6.1 Model

A priority queue is a data structure that allows at least the following two operations: `insert`, which does the obvious thing; and `deleteMin`, which finds, returns, and removes the minimum element in the priority queue.[1] The `insert` operation is the equivalent of `enqueue`, and `deleteMin` is the priority queue equivalent of the queue's `dequeue` operation.

[1] The C++ code provides two versions of `deleteMin`. One removes the minimum; the other removes the minimum and stores the removed value in an object passed by reference.

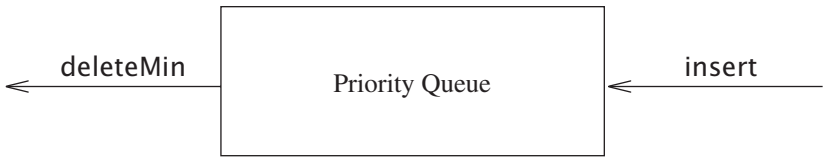

Figure 6.1 Basic model of a priority queue

As with most data structures, it is sometimes possible to add other operations, but these are extensions and not part of the basic model depicted in Figure 6.1.

Priority queues have many applications besides operating systems. In Chapter 7, we will see how priority queues are used for external sorting. Priority queues are also important in the implementation of **greedy algorithms**, which operate by repeatedly finding a minimum; we will see specific examples in Chapters 9 and 10. In this chapter, we will see a use of priority queues in discrete event simulation.

6.2 Simple Implementations

There are several obvious ways to implement a priority queue. We could use a simple linked list, performing insertions at the front in $O(1)$ and traversing the list, which requires $O(N)$ time, to delete the minimum. Alternatively, we could insist that the list be kept always sorted; this makes insertions expensive ($O(N)$) and `deleteMin`s cheap ($O(1)$). The former is probably the better idea of the two, based on the fact that there are never more `deleteMin`s than insertions.

Another way of implementing priority queues would be to use a binary search tree. This gives an $O(\log N)$ average running time for both operations. This is true in spite of the fact that although the insertions are random, the deletions are not. Recall that the only element we ever delete is the minimum. Repeatedly removing a node that is in the left subtree would seem to hurt the balance of the tree by making the right subtree heavy. However, the right subtree is random. In the worst case, where the `deleteMin`s have depleted the left subtree, the right subtree would have at most twice as many elements as it should. This adds only a small constant to its expected depth. Notice that the bound can be made into a worst-case bound by using a balanced tree; this protects one against bad insertion sequences.

Using a search tree could be overkill because it supports a host of operations that are not required. The basic data structure we will use will not require links and will support both operations in $O(\log N)$ worst-case time. Insertion will actually take constant time on average, and our implementation will allow building a priority queue of N items in linear time, if no deletions intervene. We will then discuss how to implement priority queues to support efficient merging. This additional operation seems to complicate matters a bit and apparently requires the use of a linked structure.

6.3 Binary Heap

The implementation we will use is known as a **binary heap**. Its use is so common for priority queue implementations that, in the context of priority queues, when the word *heap* is used without a qualifier, it is generally assumed to be referring to this implementation of the data structure. In this section, we will refer to binary heaps merely as *heaps*. Like binary search trees, heaps have two properties, namely, a structure property and a heap-order property. As with AVL trees, an operation on a heap can destroy one of the properties, so a heap operation must not terminate until all heap properties are in order. This turns out to be simple to do.

6.3.1 Structure Property

A heap is a binary tree that is completely filled, with the possible exception of the bottom level, which is filled from left to right. Such a tree is known as a **complete binary tree**. Figure 6.2 shows an example.

It is easy to show that a complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes. This implies that the height of a complete binary tree is $\lfloor \log N \rfloor$, which is clearly $O(\log N)$.

An important observation is that because a complete binary tree is so regular, it can be represented in an array and no links are necessary. The array in Figure 6.3 corresponds to the heap in Figure 6.2.

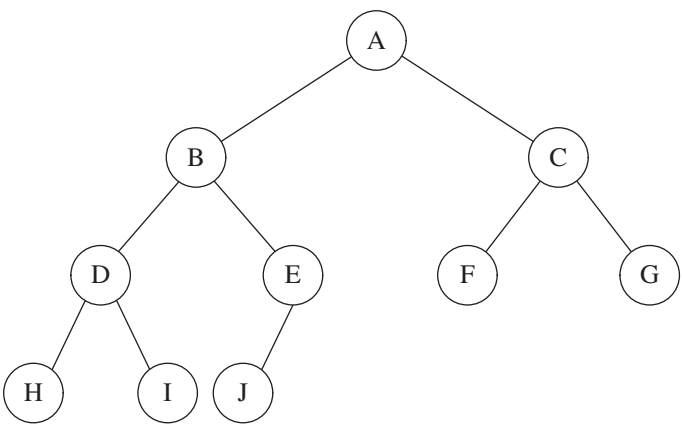

Figure 6.2 A complete binary tree

	A	B	C	D	E	F	G	H	I	J			
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Figure 6.3 Array implementation of complete binary tree

```
template <typename Comparable>
class BinaryHeap
{
  public:
    explicit BinaryHeap( int capacity = 100 );
    explicit BinaryHeap( const vector<Comparable> & items );

    bool isEmpty( ) const;
    const Comparable & findMin( ) const;

    void insert( const Comparable & x );
    void insert( Comparable && x );
    void deleteMin( );
    void deleteMin( Comparable & minItem );
    void makeEmpty( );

  private:
    int                currentSize;  // Number of elements in heap
    vector<Comparable> array;        // The heap array

    void buildHeap( );
    void percolateDown( int hole );
};
```

Figure 6.4 Class interface for priority queue

For any element in array position i, the left child is in position $2i$, the right child is in the cell after the left child $(2i + 1)$, and the parent is in position $\lfloor i/2 \rfloor$. Thus, not only are links not required, but the operations required to traverse the tree are extremely simple and likely to be very fast on most computers. The only problem with this implementation is that an estimate of the maximum heap size is required in advance, but typically this is not a problem (and we can resize if needed). In Figure 6.3 the limit on the heap size is 13 elements. The array has a position 0; more on this later.

A heap data structure will, then, consist of an array (of `Comparable` objects) and an integer representing the current heap size. Figure 6.4 shows a priority queue interface.

Throughout this chapter, we shall draw the heaps as trees, with the implication that an actual implementation will use simple arrays.

6.3.2 Heap-Order Property

The property that allows operations to be performed quickly is the **heap-order property**. Since we want to be able to find the minimum quickly, it makes sense that the smallest element should be at the root. If we consider that any subtree should also be a heap, then any node should be smaller than all of its descendants.

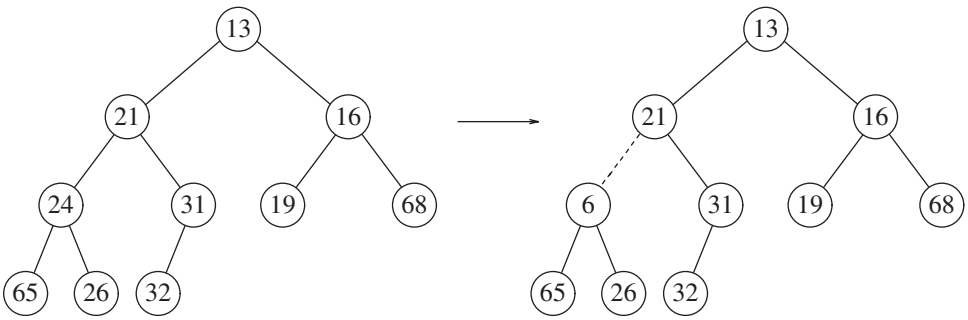

Figure 6.5 Two complete trees (only the left tree is a heap)

Applying this logic, we arrive at the heap-order property. In a heap, for every node X, the key in the parent of X is smaller than (or equal to) the key in X, with the exception of the root (which has no parent).[2] In Figure 6.5 the tree on the left is a heap, but the tree on the right is not (the dashed line shows the violation of heap order).

By the heap-order property, the minimum element can always be found at the root. Thus, we get the extra operation, `findMin`, in constant time.

6.3.3 Basic Heap Operations

It is easy (both conceptually and practically) to perform the two required operations. All the work involves ensuring that the heap-order property is maintained.

`insert`

To insert an element X into the heap, we create a hole in the next available location, since otherwise, the tree will not be complete. If X can be placed in the hole without violating heap order, then we do so and are done. Otherwise, we slide the element that is in the hole's parent node into the hole, thus bubbling the hole up toward the root. We continue this process until X can be placed in the hole. Figure 6.6 shows that to insert 14, we create a hole in the next available heap location. Inserting 14 in the hole would violate the heap-order property, so 31 is slid down into the hole. This strategy is continued in Figure 6.7 until the correct location for 14 is found.

This general strategy is known as a **percolate up;** the new element is percolated up the heap until the correct location is found. Insertion is easily implemented with the code shown in Figure 6.8.

[2] Analogously we can declare a (*max*) heap, which enables us to efficiently find and remove the maximum element by changing the heap-order property. Thus, a priority queue can be used to find *either* a minimum or a maximum, but this needs to be decided ahead of time.

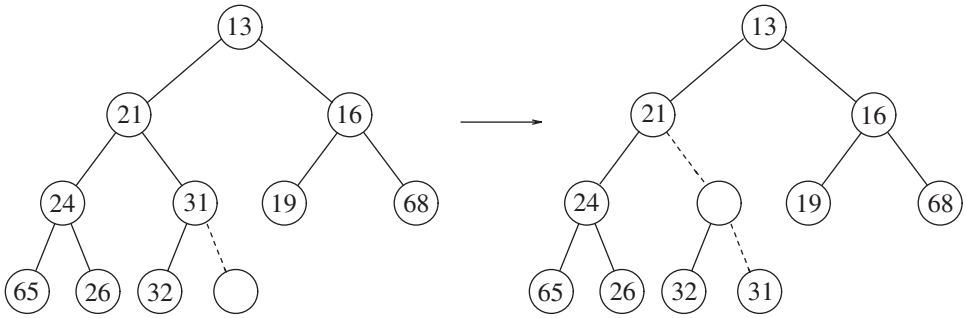

Figure 6.6 Attempt to insert 14: creating the hole, and bubbling the hole up

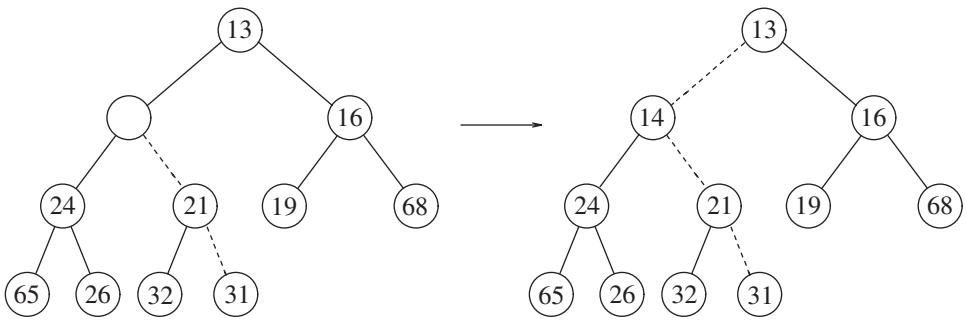

Figure 6.7 The remaining two steps to insert 14 in previous heap

```
/**
 * Insert item x, allowing duplicates.
 */
void insert( const Comparable & x )
{
    if( currentSize == array.size( ) - 1 )
        array.resize( array.size( ) * 2 );

        // Percolate up
    int hole = ++currentSize;
    Comparable copy = x;

    array[ 0 ] = std::move( copy );
    for( ; x < array[ hole / 2 ]; hole /= 2 )
        array[ hole ] = std::move( array[ hole / 2 ] );
    array[ hole ] = std::move( array[ 0 ] );
}
```

Figure 6.8 Procedure to insert into a binary heap

We could have implemented the percolation in the `insert` routine by performing repeated swaps until the correct order was established, but a swap requires three assignment statements. If an element is percolated up d levels, the number of assignments performed by the swaps would be $3d$. Our method uses $d + 1$ assignments.

If the element to be inserted is the new minimum, it will be pushed all the way to the top. At some point, `hole` will be 1 and we will want to break out of the loop. We could do this with an explicit test, or we can put a copy of the inserted item in position 0 in order to make the loop terminate. We elect to place X into position 0.

The time to do the insertion could be as much as $O(\log N)$, if the element to be inserted is the new minimum and is percolated all the way to the root. On average, the percolation terminates early; it has been shown that 2.607 comparisons are required on average to perform an insert, so the average `insert` moves an element up 1.607 levels.

`deleteMin`

`deleteMin`s are handled in a similar manner as insertions. Finding the minimum is easy; the hard part is removing it. When the minimum is removed, a hole is created at the root. Since the heap now becomes one smaller, it follows that the last element X in the heap must move somewhere in the heap. If X can be placed in the hole, then we are done. This is unlikely, so we slide the smaller of the hole's children into the hole, thus pushing the hole down one level. We repeat this step until X can be placed in the hole. Thus, our action is to place X in its correct spot along a path from the root containing *minimum* children.

In Figure 6.9 the left figure shows a heap prior to the `deleteMin`. After 13 is removed, we must now try to place 31 in the heap. The value 31 cannot be placed in the hole, because this would violate heap order. Thus, we place the smaller child (14) in the hole, sliding the hole down one level (see Fig. 6.10). We repeat this again, and since 31 is larger than 19, we place 19 into the hole and create a new hole one level deeper. We then place 26 in the hole and create a new hole on the bottom level since, once again, 31 is too large. Finally, we are able to place 31 in the hole (Fig. 6.11). This general strategy is known as a **percolate down.** We use the same technique as in the `insert` routine to avoid the use of swaps in this routine.

A frequent implementation error in heaps occurs when there are an even number of elements in the heap, and the one node that has only one child is encountered. You must

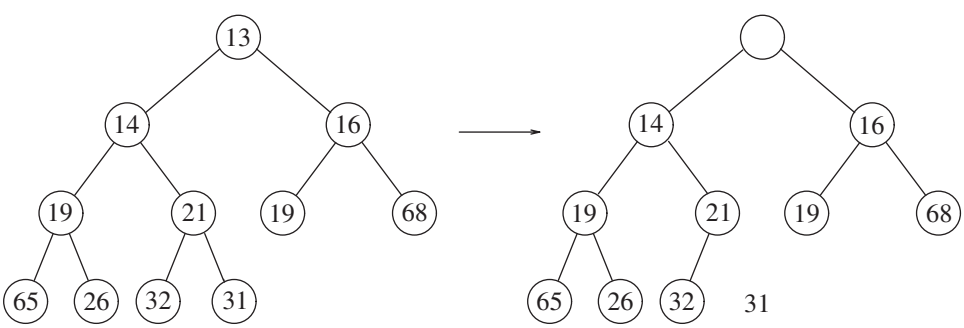

Figure 6.9 Creation of the hole at the root